

UNIT STUDY NOTES

Python Core Fundamentals

The syntactic and semantic foundation everything else assumes: how Python evaluates expressions and statements, how values are printed and read, how control flow is expressed, and how text is represented and manipulated.

 3 source notebooks 84 topic sections extracted 6 study sections

Unit Overview

Python is the first step for any aspiring data scientist or ML engineer. This notebook covers the **foundational grammar** of Python: writing output with `print`, the core **data types, variables** and how they bind to values, comments, **keywords & identifiers**, taking **input** from the user, **type conversion, literals**, and an overview of operators and if-else. These are the same building blocks you will see everywhere in an ML pipeline: loading data, checking dtypes, transforming values, and printing results to verify them.

Why it matters for ML: Python does not force you to write type declarations like `int a = 5`. You simply write `a = 5` and Python decides the type at runtime (dynamic typing). Understanding how types behave is exactly what you need before moving to NumPy arrays and Pandas DataFrames.

How to study: every section places the **theory first** (what + why), followed immediately by its **code cell** (how). Run each cell and observe the output. Theory read together with its output is what makes the concept stick.

Roadmap of this session:

1. Python Output — `print()` with `sep` / `end`
2. Data Types — int, float, complex, bool, str, list, tuple, set, dict
3. Variables — static vs dynamic typing/binding
4. Comments — notes inside code
5. Keywords & Identifiers — reserved words and naming rules
6. User Input — `input()` and the always-returns-string rule
7. Type Conversion — implicit vs explicit casting
8. Literals — binary, hex, octal, raw strings, None
9. Operators — a 30-second tour of all families (detailed in Session 2)
10. If-Else — the basic idea of branching (detailed in Session 2)

Let us begin with topic 1 — **Python Output**.

Understanding `print()`

What it is

`print()` outputs values to standard output.

Why

For displaying results and debugging.

How it works

Converts args to strings, joins with `sep` (default space), ends with `end` (default newline), writes to `stdout`.

Parameters

- `sep`: separator between values
- `end`: terminator

Common mistakes

Confusing `sep/end`, expecting return (`None`).

Edge cases

`print()` with no args gives newline.

Welcome to the second session of your Python journey. You can already store values and run simple code, but real programs are built from decisions and repetition. This session gives you the two superpowers that turn a script into a program: operators, which let you compute and compare values, and control flow, which lets you choose what runs next and repeat work without copying code.

Here is the road map for this notebook:

1. **Arithmetic operators** — `+`, `-`, `*`, `/`, `//`, `%`, `**` for everyday math.
2. **Relational operators** — compare values and produce `True` or `False`.
3. **Logical operators** — combine conditions with `and`, `or`, and `not`.
4. **Bitwise operators** — work directly on the binary digits of integers.
5. **Assignment operators** — compound shorthand such as `+=`, `-=`, `%=`.
6. **Membership operators** — test whether a value lives inside a container with `in` / `not in`.
7. A small **program** that combines several of these ideas at once.
8. **If-else branches** — indentation-based decision making, nested `if` flows, and menu-driven programs.
9. **Modules** — reuse ready-made functionality from the standard library (`math`, `random`, `keyword`, `datetime`).
10. **Loops** — `while` and `for`, loops with `else`, nested loops, and the loop-control trio `break`, `continue`, `pass`.

Why does all of this matter? Every program you will ever write — a calculator, a login screen, an e-commerce checkout, a data pipeline — is built from three ingredients: values, computations, and repetition. Operators do the computations; conditions select the right branch; loops repeat a rule until the job is done. Once you can combine all three fluently, you stop writing toy programs and start building real ones.

As you work through this notebook, run every code cell and match each line of output to the rule that produced it. When you meet a program, pause and predict the result before executing — prediction is the fastest way to build intuition. By the end you will be comfortable reading and writing code that branches and loops, which is exactly the foundation you need before moving on to strings, lists, and functions.

Happy learning!

What/Why/How

Operators perform operations on operands. They exist to express computations concisely. Understanding precedence and associativity prevents bugs. Type coercion rules matter for mixed types.

When to use

Arithmetic/logical/comparison for control flow and calculations.

Common mistakes

Operator precedence confusion (use parentheses), equality vs identity confusion later.

Edge cases

Division by zero, integer vs float, short-circuit evaluation.

Session 3 shifts from numbers and loops to **text**. Computers deal with text everywhere — file names, emails, log messages, JSON, and the return value of a language model. In Python, all text lives inside a built-in type called **string** (`str`).

What is a string? A string is an immutable, ordered sequence of characters enclosed in quotes (`'a'`, `"a"`, or `'''a'''` for multi-line). 'Immutable' means once created, a string can never be changed in place — every method returns a NEW string. 'Ordered' means every character has a position (index) that we can read with `s[i]`.

Why it matters in AI/ML: text is the raw fuel for NLP. Cleaning it — lowercasing, removing punctuation, splitting into words, measuring length — is done entirely with string methods. `split`, `strip`, `replace`, `count`, and `join` are the bread-and-butter operations of every text pipeline.

What this session covers (in order):

1. Creating strings
2. Indexing & slicing
3. Immutability & editing
4. Operations on strings (concatenation, repetition, comparisons, iteration, membership)

5. Common functions — len / max / min / sorted
6. Case methods — capitalize / title / upper / lower / swapcase
7. Search — count / find / index
8. Checks — endswith / startswith / isalnum / isalpha / isdigit / isidentifier
9. format()
10. Split / join
11. Replace
12. Strip
13. Eight example programs

At the very top, a few leftover topics from Session 2 (population problem, sequence sum, patterns, loop control) appear as quick warm-up practice.

Study each topic: read the theory, then run the code right below it.

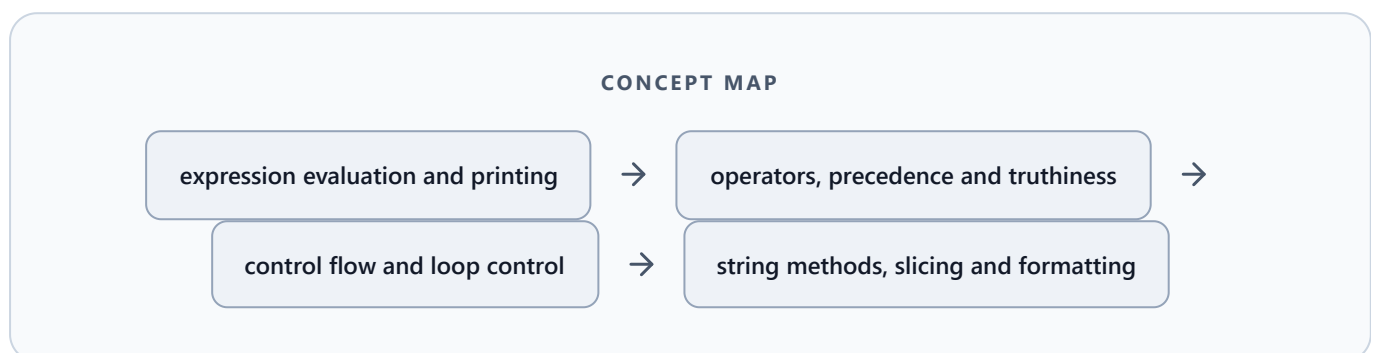
Prerequisites

→ Jupyter familiarity from Unit 00

What you will be able to do

- ✓ Use print, input and f-strings with an accurate model of evaluation
- ✓ Apply every operator category with correct precedence and truthiness
- ✓ Write reliable if/elif/else and loop logic, including break/continue/pass
- ✓ Manipulate strings using slicing, methods and formatting rather than ad-hoc code

What This Unit Covers



The sections below follow this arc, from foundational semantics to the engineering decisions that follow from them.

Core Concepts

Python Basics

1.1 print() — basics, quoting & case sensitivity

What it is: `print()` is Python's built-in function that displays values as text on the console.

Why: without output you cannot see what a program computed. Debugging, logging, and printing metrics in ML all start with `print()`.

How it works: `print(value1, value2, ..., sep=' ', end='\n')`. Every value is converted with an internal `str()` before being written. Strings must be written in quotes; without quotes `print(Ravnoor Singh)` raises a `SyntaxError/NameError`. Python is **case sensitive** — `print` works, `Print` does not.

Key parameters:

- `sep` — the character placed between values (default: space)
- `end` — the character placed after the last value (default: `\n`)

Small example: the code prints `'Hello World'` first, then intentionally runs `print(Ravnoor Singh)` without quotes to demonstrate the quoting rule.

Python Output

What is print()

`print()` writes objects to standard output.

Why it exists

To display results and debug.

How it works

Arguments converted to strings, joined by `sep`, ended by `end`.

Parameters

- `sep`: default space
- `end`: default newline

Common mistakes

Confusing `sep` vs `end`; expecting return (returns `None`).

PYTHON

```
# Python is a case sensitive language
print('Hello World')
```

PYTHON

```
print('Ravnoor Singh')
```

1.2 Multiple values & the sep parameter

What it is: `print()` can output several values in one call, and `sep` sets the character placed between those values.

Why: showing multiple data points together (like all columns of one row) is cleaner in a single call.

How it works: `print('Hello', 1, 4.5, True)` joins the values with spaces since the default is `sep=' '`. With `sep='/'` the output becomes `Hello/1/4.5/True`. Types can be mixed freely — Python converts each one to a string.

Small example: both calls below use the same values; only `sep` differs, so the output changes. Observe the difference.

PYTHON

```
print(7)
```

PYTHON

```
print(7.7)
```

1.3 The sep & end pair

`sep` joins multiple values; `end` decides what comes after the last value. Together they fully control the shape of your output line.

PYTHON

```
print('Hello', 1, 4.5, True, sep='/')
```

1.3 The end parameter & newline behaviour

What it is: `end` decides what gets printed after the last value. The default is `\n` (newline), so the next print starts on a new line.

Why needed: sometimes we want two prints on the same line, for example when showing progress steps.

How it works: `print('hello', end='-')` prints `hello-` without a newline. The following `print('world')` continues on the same line, producing `hello-world`.

Small example: the code below shows `print('hello')` + `print('world')` on two lines, then the same with `end='-'` merging them.

PYTHON

```
print('hello')
print('world')
```

PYTHON

```
print('hello',end=' - ')
print('world')
```

2. Data Types

What/Why

Variables reference objects in memory. Data types determine operations allowed and behavior.

Semantics

Python is dynamically typed. Assignment binds names to objects; mutability matters.

Common mistakes

Confusing assignment vs comparison, mutability side effects.

2.1 Numeric types — int & float

What it is: `int` stores whole numbers (positive, negative, zero, of any size), `float` stores decimal numbers like 8.55 or 1.5e2.

Why: numbers are the actual data of machine learning — features, weights, and loss values are all numeric.

How it works: `print(8)` writes an integer. A huge float literal like `1e309` overflows to `inf` (infinity) instead of crashing, because float is not size-limited like C integers.

Small example: the code prints an int (`8`), a decimal (`8.55`) and a huge float (`1e309`); the `inf` output is the key observation.

PYTHON

```
# Integer
print(8)
# 1*10^308
print(1e309)
```

PYTHON

```
# Decimal/Float
print(8.55)
print(1.7e309)
```

2.2 bool & str — truth values and text

What it is: `bool` stores only `True` / `False`; `str` stores any text.

Why: conditions (True/False) and textual data (names, labels) are everywhere in ML — classification labels are strings too.

How it works: `bool` is actually a subclass of `int` (`True==1`, `False==0`). Strings are written inside quotes and are immutable. Both display as text when printed.

Key point: everything in Python is an object, and `type(True)` reveals its class.

Small example: below we print `True`, `False`, and a string.

PYTHON

```
# Boolean
print(True)
print(False)
```

PYTHON

```
# Text/String
print('Hello World')
```

2.3 complex, collections & type()

What it is: `complex` holds numbers with a real and imaginary part ($5+6j$). Collections are `list` (mutable, ordered), `tuple` (immutable, ordered), `set` (unique values) and `dict` (key:value pairs). `type()` returns the type of any value.

Why: complex numbers appear in linear algebra and FFT; lists and dicts are the building blocks of Pandas and NumPy; `type()` is used for debugging and validation.

How it works: in $5+6j$ the `j` is the imaginary unit — use `.real` and `.imag` to access the parts. `type([1,2,3])` returns `<class 'list'>`.

Small example: the code prints an example of each type — complex, list, tuple, set, dict — and finally a `type()` check.

PYTHON

```
# complex
print(5+6j)
```

2.4 Data Types — at a glance

Key types table:

TYPE	EXAMPLE	MEANING
<code>int</code>	<code>8</code>	whole number
<code>float</code>	<code>8.55</code>	decimal number
<code>bool</code>	<code>True</code>	truth values
<code>str</code>	<code>'Hello'</code>	text
<code>complex</code>	<code>5+6j</code>	real + imaginary
<code>list</code>	<code>[1,2,3]</code>	ordered, mutable
<code>tuple</code>	<code>(1,2,3)</code>	ordered, immutable
<code>set</code>	<code>{1,2,3}</code>	unordered, unique
<code>dict</code>	<code>{'name': 'X'}</code>	key:value map

Remember: use `type(x)` to ask what type a value has. Every literal's type is shown as a class object like `<class 'int'>`.

PYTHON

```
# List -> C -> Array
print([1,2,3,4,5])
```

PYTHON

```
# Tuple
print((1,2,3,4,5))
```

3.1 Static vs Dynamic Typing

What it is: typing decides when a variable's type is fixed. In static languages (C/C++) you must declare the type at compile time (`int a = 5`); in Python the type is decided at runtime (`a = 5`).

Why: static typing catches errors earlier but makes code longer. Python's dynamic typing keeps ML prototyping fast and clean.

How it works: `a = 5` creates an integer object and tags it `a`. Writing `int a = 5` raises a `SyntaxError` because Python does not support that C-style syntax.

Small example: the code below runs `a = 5` and `b = 5` fine; the static-style `int a = 5` produces an error.

PYTHON

```
# Static Vs Dynamic Typing
# Static Vs Dynamic Binding
# stylish declaration techniques
```

PYTHON

```
# C/C++
name = 'nitish'
print(name)

a = 5
b = 6

print(a + b)
```

3.2 Dynamic Binding & assignment styles

What it is: binding is how a name attaches to a value. Dynamic binding means a variable can change its type while the program runs.

Why: Python allows `a = 5` followed by `a = 'hello'`; C/C++ rejects a type change.

Assignment styles:

- separate lines: `a = 1; b = 2; c = 3`
- parallel / tuple unpacking: `a, b, c = 1, 2, 3`
- chained: `a = b = c = 5` (all get 5)

How it works: a variable is a label pointing to an object; a new assignment moves the label to a new object.

Small example: the code demonstrates dynamic binding (`a = 5` becomes `a = 'nitish'`) and prints the results of all three assignment styles.

PYTHON

```
# Dynamic Binding
a = 5
print(a)
a = 'nitish'
print(a)

# Static Binding
int a = 5
```

PYTHON

```
a = 1
b = 2
c = 3
print(a, b, c)
```

4. Comments — notes inside your code

What it is: comments are text that Python **completely ignores**; they exist only for human readers.

Why: to explain complex logic, to document what and why, and to temporarily disable code while debugging.

How it works: everything from `#` to the end of the line is a comment. They work on their own line or after code: `b = 6 # inline comment`.

Remember: comments explain *why*, not *what*. Good comments document your intent.

Small example: the code below contains `#` lines; they never execute, only `print(a+b)` produces output.

PYTHON

```
# this is a comment
# second line
a = 4
b = 6 # like this
# second comment
print(a+b)
```

5.1 Keywords — reserved words

What it is: keywords are Python's **reserved words** (`if`, `for`, `class`, `return`, ...). You cannot use them as variable/identifier names.

Why: these words drive the language's syntax, so Python reserves their meaning.

How it works: Python has 35 keywords. To list them:

Small example: `keyword.kwlist` is run below and prints the full list of reserved words.

PYTHON

```
# Keywords
import keyword
keyword.kwlist
```

PYTHON

```
import keyword
print(keyword.kwlist)
```

5.2 Identifiers — naming rules

What it is: identifiers are the names you create (variables, functions).

Rules:

1. Only letters, digits, and underscore `_` are allowed
2. Cannot start with a digit (`1name` is invalid)
3. Cannot be a keyword
4. Case sensitive (`name` != `Name`)
5. A lone `_` is a valid identifier (used for unused values)

Conventions (PEP 8): variables use `snake_case`; classes use `PascalCase`.

Small example: the code creates `name1` and `_` as valid identifiers.

PYTHON

```
# Identifiers
# You can't start with a digit
name1 = 'Nitish'
print(name1)
# You can use special chars -> _
_ = 'ntiish'
print(_)
# identifiers can not be keyword
```

6. User Input — the input() function

What it is: `input(prompt)` reads one line from the user.

Why: ML programs often ask the user for hyper-parameters or file paths.

CRITICAL RULE: `input()` always returns a string, even if the user types digits.

How it works: `fnum = int(input('enter first number'))` — the `int()` cast is mandatory; without it, `'45' + '34'` becomes the string `'4534'` (concatenation), not the number 79.

Small example: the code reads two numbers, casts them with `int()`, and prints their sum and their types.

PYTHON

```
# Static Vs Dynamic
input('Enter Email')
```

PYTHON

```
# take input from users and store them in a variable
fnum = int(input('enter first number'))
snum = int(input('enter second number'))
print(type(fnum), type(snum))
```

7.1 Implicit Type Conversion

What it is: Python converts a type **automatically** whenever it is safe.

Why: in `5 + 5.6` an int and a float are mixed, so Python widens the smaller type (int) to the larger one (float).

How it works: narrow → wide. `int + float → float`, done silently.

IMPORTANT: `4 + '4'` raises a **TypeError** — Python will NOT auto-convert a string to an int.

Small example: `5 + 5.6` works (implicit float), but `4 + '4'` errors — the golden rule: int + string never works until you cast.

PYTHON

```
# Implicit Vs Explicit
print(5+5.6)
print(type(5),type(5.6))

print(4 + '4')
```

7.2 Explicit Type Conversion (Casting)

What it is: you force the conversion yourself using constructors: `int()`, `float()`, `str()`, `bool()`.

Why: input strings need to become numbers, and numbers need to become display-ready strings.

How it works:

- `int('1234')` → 1234 (digit strings are fine)
- `float(4)` → 4.0
- `str(5)` → '5'
- `int(4+5j)` → **TypeError** (a complex number cannot become an int)

Small example: the code shows str→int, int→str, and float conversions. Study the outputs to see each type change.

PYTHON

```
# Explicit
# str -> int
#int(4+5j)

# int to str
str(5)

# float
float(4)
```

8.1 Numeric literals — binary / octal / hex / float

What it is: a literal is a value written directly in the source code.

Numeric prefixes (exam favourite):

- Binary: `0b1010` → 10
- Octal: `0o310` → 200
- Hex: `0x12c` → 300
- Float: `10.5`; scientific `1.5e2` → 150.0; `1.5e-3` → 0.0015

Small example: below, a, b, c, d hold the four number systems and are printed together — Python always displays them in decimal.

PYTHON

```

a = 0b1010 #Binary Literals
b = 100 #Decimal Literal
c = 0o310 #Octal Literal
d = 0x12c #Hexadecimal Literal

#Float Literal
float_1 = 10.5
float_2 = 1.5e2 # 1.5 * 10^2
float_3 = 1.5e-3 # 1.5 * 10^-3

#Complex Literal
x = 3.14j

print(a, b, c, d)
print(float_1, float_2, float_3)
print(x, x.imag, x.real)

```

8.2 String literals, complex literals & None

What it is:

- String literals: `'single'`, `"double"`, `'''multi-line'''`, `u"..."` (unicode), `r"..."` (raw — backslashes stay literal)
- Complex literal: `3.14j` with `.real` / `.imag`
- `None` — "nothing / not assigned"

Why: raw strings matter for regex and paths; None is how Python represents missing values.

Small example: the code prints all string literal styles, the real/imag parts of `3.14j`, and a small program using `None`.

PYTHON

```

# binary
x = 3.14j
print(x.imag)

```

PYTHON

```

string = 'This is Python'
strings = "This is Python"
char = "C"
multiline_str = """This is a multiline string with more than one line code."""
unicode = u"\U0001f600\U0001f606\U0001f923"
raw_str = r"raw \n string"

print(string)
print(strings)
print(char)
print(multiline_str)
print(unicode)
print(raw_str)

```

8.3 Bool as int & None

What it is: `True` is numeric internally — `True + 4 = 5` and `False + 10 = 10`.

Why: bool is a subclass of int, so it participates in arithmetic.

How it works: `a = True + 4` becomes 5. `b = False + 10` becomes 10. `None` represents "no value yet".

Small example: the code below prints the results of boolean arithmetic.

PYTHON

```

a = True + 4
b = False + 10

print("a:", a)
print("b:", b)

```

PYTHON

```

k = None
a = 5
b = 6
print('Program exe')

```

9. Operators — Overview

What it is: operators are symbols that perform operations on values (operands).

Families: Arithmetic (`+` `-` `*` `/` `//` `%` `**`), Relational (`==` `!=` `>` `<` `>=` `<=`), Logical (`and` `or` `not`), Bitwise (`&` `|` `^` `~` `<<` `>>`), Assignment (`=` `+=` `-=` `...`), Membership (`in` `not in`).

Why: without operators there is no way to process data — every ML calculation runs on these symbols.

Small example: the code lists representative operators of each family. Session 2 covers every family in detail.

Priority (short): `()` `>` `**` `>` `-x` `>` `* / // %` `>` `+` `-` `>` comparison `>` `not` `>` `and` `>` `or`.

PYTHON

```
# Arithmetic
# Relational
# Logical
# Bitwise
# Assignment
# Membership
```

10. If-Else — an Overview of branching

What it is: branching means "run this block only when a condition is true, otherwise that one".

Why: every rule-based system makes decisions — thresholds, category grouping, data-cleaning rules.

How it works:

Golden rule: INDENTATION defines the block (4 spaces) — Python has no curly braces. Full programs are covered in Session 2.

PYTHON

```
if condition:
    # block (runs when True)
elif condition2:
    # block 2
else:
    # block 3
```

Summary

- `print(v1, v2, sep=..., end=...)` — the basic tool for output
- Data types: int, float, complex, bool, str, list, tuple, set, dict — check with `type()`
- Python uses **dynamic typing + binding** — `a = 5` then `a = 'x'` are both valid
- Comments start with `#`; keywords are reserved; identifiers follow naming rules
- `input()` **always returns a string** — cast with `int()` for numbers
- Conversions: implicit (automatic) vs explicit (`int()` / `float()` / `str()`)
- Literals: `0b/0o/0x`, raw strings, `None`; `True` behaves like 1
- Operators come in 6 families; if-else relies on indentation

Common mistakes: forgetting quotes (`print(var)` vs `print('var')`), forgetting the `int()` cast on input, or writing C-style `int a = 5`.

Next: Session 2 — Operators in full detail + if-else + loops (for/while).

This notebook covered the operator families Python provides and the control-flow tools that bring programs to life. On the operator side you met arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`), relational operators that return booleans, logical operators with short-circuit behaviour, bitwise operators that work on binary digits, compound assignment operators like `+=` and `%=`, and membership tests with `in` / `not in` — then

used them to build a digit-sum program. On the control-flow side you used `if` / `elif` / `else` with indentation as the block rule, built a login flow and a menu-driven calculator, imported standard-library modules such as `math`, `random`, `keyword`, and `datetime`, and finally mastered `while` and `for` loops, including loops with an `else` clause, nested loops for pattern printing, and the `break`, `continue`, and `pass` statements. The population and sequence-sum programs showed how a loop plus a small update rule solves problems that would be tedious by hand. With operators and control flow under your belt, you are ready to tackle collections — strings, lists, and dictionaries — in the next sessions.

- **Strings** are ordered, **immutable** sequences of characters — every method returns a NEW string
- **Indexing:** `s[0]` from the left, `s[-1]` from the right; slicing `s[start:stop:step]` grabs substrings and `[::-1]` reverses
- **Arithmetic:** `+` concatenates, `*` repeats; compare strings with `>` lexically by ASCII code
- **Empty string** `''` is falsy — `and/or` return operand values, not booleans
- **Search:** `count()`, `find()` (returns -1 when absent), `index()` (raises `ValueError`)
- **Checks:** `endswith/startswith`, and the `is...` families — `isalnum`, `isalpha`, `isdigit`, `isidentifier`
- **Cleaning tools:** `lower/capitalize/title/upper/swapcase`, `strip` (also `lstrip/rstrip`), `replace`, `split` → list, `join` ← rebuilding
- `format()` lets placeholders `{0}` `{1}` reorder arguments; f-strings are the modern alternative
- **Eight programs** tie everything together: manual len, email username, frequency, remove char, palindrome, word count, title case, int→string

Next: Session 4 — Lists (mutable sequences, slicing, methods).

Operators in Python

Arithmetic Operators

What it is: the normal math operators — `+` (add), `-` (subtract), `*` (multiply), `/` (true divide → float), `//` (floor divide → integer), `%` (remainder/modulus), `**` (power).

Why it matters: nearly every numeric pipeline — scores, deltas, normalization, splittling digits — is built on these five symbols plus power.

Run the code cell below and match each line of output to its operator.

PYTHON

```
# Arithmetic Operators
print(5+6)

print(5-6)

print(5*6)

print(5/2)

print(5//2)

print(5%2)

print(5**2)
```

PYTHON

```
print(5 + 6)    # 11    addition
print(10 - 3)   # 7     subtraction
print(5 / 2)    # 2.5   true division
print(5 // 2)   # 2     floor division
print(5 % 2)    # 1     remainder
print(5 ** 2)   # 25    power
```

Relational (Comparison) Operators

What it is: operators that compare two values and give a **boolean** (`True` / `False`): `>`, `<`, `>=`, `<=`, `==` (equal), `!=` (not equal).

Do not confuse `=` **with** `==`: `=` assigns a value; `==` asks 'are these equal?'.

Why it matters: comparisons are the engine of every condition — thresholds, filters, and anomaly checks all reduce to `True` / `False`.

Run the code cell below and read each output against its predicate.

PYTHON

```
# Relational Operators
print(4>5)

print(4<5)

print(4>=4)

print(4<=4)

print(4==4)

print(4!=4)
```

PYTHON

```
print(4 > 5)      # False
print(4 < 5)      # True
print(4 >= 4)     # True
print(4 <= 4)     # True
print(4 == '4')  # False - no automatic type conversion!
print(4 != 5)    # True
```

Logical Operators

What it is: combine truth values: `and`, `or`, `not`. They evaluate operands and **return the operand value** (short-circuit), not always a bool.

Truthiness: falsy = `0`, `0.0`, `' '`, `[]`, `{}`, `None`, `False`; everything else is truthy.

Why it matters: compound business rules ('category is X AND price > 50') are written with logical operators.

Run the code cell below — note that `and` / `or` print operand values.

PYTHON

```
# Logical Operators
print(1 and 0)

print(1 or 0)

print(not 1)
```

PYTHON

```
print(1 and 0)    # 0      - first falsy wins
print(1 or 0)     # 1      - first truthy wins
print(not 1)      # False  - always a bool
```

Bitwise Operators

What it is: operators that work on the **binary digits** of integers: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (left shift), `>>` (right shift).

Why it matters: flags, permissions, and bit-mask tricks use these; they are also the fastest possible math on integers.

Run the code cell below and verify each binary result.

PYTHON

```
# Bitwise Operators

# bitwise and
print(2 & 3)

# bitwise or
print(2 | 3)

# bitwise xor
print(2 ^ 3)

print(~3)

print(4 >> 2)

print(5 << 2)
```

PYTHON

```
print(2 & 3)      # 2   bits: 10 & 11 = 10
print(2 | 3)      # 3   bits: 10 | 11 = 11
print(2 ^ 3)      # 1   bits: 10 ^ 11 = 01
print(~3)         # -4  flips bits (two's complement)
print(4 >> 2)      # 1   shift right = divide by 2^n
print(5 << 2)      # 20  shift left  = multiply by 2^n
```

Assignment Operators

What it is: `=` stores a value; compound forms (`+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`, `&=`, `|=`, `^=`, `>>=`, `<<=`) combine an operation with assignment.

Important: Python has **no** `a++` / `++a` — use `a += 1`.

Run the code cell below — the commented example shows `a %= 2` rewriting the variable in place.

PYTHON

```
# Assignment Operators

# =
# a = 2

a = 2

# a = a % 2
a %= 2

# a++ ++a

print(a)
```

PYTHON

```
a = 2          # assignment
a = a + 1      # becomes 3
a += 1         # ALSO 3 - shorthand
a %= 2         # a = a % 2 → 3 % 2 = 1
```

Membership Operators

What it is: `in` / `not in` check whether a value exists inside a container (string, list, tuple, set, dict keys).

Details: string membership is case-sensitive and checks substrings; dict membership checks **keys**.

Run the code cell below to watch `not in` and `in` in action.

PYTHON

```
# Membership Operators

# in/not in

print('D' not in 'Delhi')

print(1 in [2,3,4,5,6])
```

PYTHON

```
print('D' not in 'Delhi')  # False - D IS present
print(1 in [2, 3, 4])      # False
print('x' in 'mumbai')     # False
```

Program — sum of digits of a 3-digit number

Problem: read a 3-digit number like `123` and print the sum of its digits (`1+2+3 = 6`).

The digit-extraction trick:

Repeat `% 10` to peel a digit, then `// 10` to shrink — until the number is 0. This single trick powers palindromes, reversals, and checksums.

Run the code cell below to see the full loop.

PYTHON

```
# Program - Find the sum of a 3 digit number entered by the user

number = int(input('Enter a 3 digit number'))

# 345%10 -> 5
a = number%10

number = number//10

# 34%10 -> 4
b = number % 10

number = number//10
# 3 % 10 -> 3
c = number % 10

print(a + b + c)
```

PYTHON

```
number = int(input('enter number')) # 123
last = number % 10 # 3 remainder
rest = number // 10 # 12 chop the last digit
```

If-else in Python

Branches decide what the program does next. We cover:

- `if` with **indentation** as the block rule
- nested `if` inside `if` (login flow)
- `min of 3` comparison logic
- a **menu-driven** calculator

The code cells after each topic are the practice arena.

if-else — indentation defines the block

What it is:

Rules:

1. Python has no `{ }` — indentation (traditionally 4 spaces) groups statements
2. A condition can be any expression that evaluates to True/False
3. `else` is optional; you may chain `elif` for more branches
4. Nested `if` means an `if` inside another `if` body

Why it matters: a login check — email matches AND password matches — is a nested-if flow with distinct messages for each failure.

Run the code cell below (login program).

PYTHON

```
# Login program and indentation
# email -> nitish.campusx@gmail.com
# password -> 1234

email = input('enter email')
password = input('enter password')

if email == 'nitish.campusx@gmail.com' and password == '1234':
    print('Welcome')
elif email == 'nitish.campusx@gmail.com' and password != '1234':
    # tell the user
    print('Incorrect password')
    password = input('enter password again')
    if password == '1234':
        print('Welcome,finally!')
    else:
        print('beta tumse na ho paayega!')
else:
    print('Not correct')
```

PYTHON

```
if condition:
    # indentation = block starts here
    # every indented line belongs to the if
else:
    # runs when condition is False
```

if-else — min of 3 & menu-driven program

Min of 3 numbers: compare each candidate against the other two:

`<=` (not `<`) handles ties gracefully.

Menu-driven program: show numbered options, read the user's choice, and run one task per `elif` branch — the structure behind every ATM / CLI menu.

Run the code cells below — one finds the minimum, the second is a mini calculator.

PYTHON

```
# if-else examples
# 1. Find the min of 3 given numbers
# 2. Menu Driven Program
```

PYTHON

```
# min of 3 number

a = int(input('first num'))
b = int(input('second num'))
c = int(input('third num'))

if a<b and a<c:
    print('smallest is',a)
elif b<c:
    print('smallest is',b)
else:
    print('smallest is',c)
```

Modules in Python

A **module** is a `.py` file packed with ready-made functions and variables. Instead of reinventing math, randomness, or dates, you `import` them.

Modules we use here:

- `math` — squares, logs, floors
- `keyword` — Python's reserved words
- `random` — random numbers
- `datetime` — current date and time

The code cells below import and call each one.

Modules — the standard library

Two import styles: `import math` (call as `math.sqrt`) or `from math import sqrt` (call as `sqrt` directly).

Run the code cells below: the first lists every built-in module name in your Python. The rest call one stdlib function each.

PYTHON

```
import sys
sys.stdlib_module_names
```

PYTHON

```
# math
import math

math.sqrt(196)
```

Loops in Python

Loops repeat work. **Need for loops:** instead of writing the same line 10 times, write the body once and let Python repeat it.

- **While loop** — repeats while a condition is true
- **For loop** — walks over every item of a sequence

The cells below cover both, plus the population program, the sequence-sum, pattern printing, and loop control. Run each cell after its theory.

While loop — repeat while true

Wait — is `while i < 11` enough? No: the loop checks the condition, runs the body, then re-checks. Forgetting `i += 1` = infinite loop.

Uses in this session: printing a multiplication table and summing the digits of a number (peel `%10`, chop `//10`).

Run the code cells below (table + digit sum).

PYTHON

```
# While loop example -> program to print the table
# Program -> Sum of all digits of a given number
# Program -> keep accepting numbers from users till he/she enters a 0 and then find the avg
```

PYTHON

```
number = int(input('enter the number'))

i = 1

while i<11:
    print(number, '*', i, '=', number * i)
    i += 1
```

while loop with `else`

A `while` loop may carry an `else` block that runs **once after the loop finishes normally** (the condition turned False — no `break`).

Gotcha: if a `break` exits the loop, the `else` is **skipped**.

Run the code cell below and notice the order of printed lines.

PYTHON

```
# while loop with else

x = 1

while x < 3:
    print(x)
    x += 1

else:
    print('limit crossed')
```

PYTHON

```
x = 1
while x < 3:
    print(x)
    x += 1
else:
    print('loop ended')      # prints AFTER the Loop
```

Guessing game — while in action

Why while and not for? We do not know how many tries the user needs — the loop must keep going **until** a condition is met.

Run the code cell below and play it.

PYTHON

```
# Guessing game

# generate a random integer between 1 and 100
import random
jackpot = random.randint(1,100)

guess = int(input('guess karo'))
counter = 1
while guess != jackpot:
    if guess < jackpot:
        print('galat!guess higher')
    else:
        print('galat!guess lower')

    guess = int(input('guess karo'))
    counter += 1

else:
    print('correct guess')
    print('attempts',counter)
```

PYTHON

```
import random
target = random.randint(1, 100) # secret number
guess = 0
while guess != target:
    guess = int(input('guess: '))
    if guess > target: print('too high')
    elif guess < target: print('too low')
```

For loop — iterate a sequence

For = for each. The loop visits every element of any *iterable* — list, tuple, set, dict, string, `range`.

Set gotcha: sets are unordered — the printed order may surprise you.

range is the counter: `range(start, stop, step)` e.g. `range(10, 0, -1)` counts 10, 9, ..., 1.

Run the code cells below — first a set demo, then an examples cell.

PYTHON

```
# For Loop demo

for i in {1,2,3,4,5}:
    print(i)
```

PYTHON

```
for i in {1, 2, 3, 4, 5}:    # set
    print(i)
```

Program — population problem

The current population of a town is **10000** and it grows 10% every year. Print the population at the end of each of the last 10 years.

Population program

Solution trick: growth of 10% per year means $next\ year = current \times 1.1$. To go **backwards** in time, undo that multiplication:

Why `/= 1.1` and NOT `-= 10%`? A 10% decrease is not the inverse of a 10% increase — percentages multiply; subtracting drifts further from the true value every year.

Run the code cells below — the first is the session code, the second the expected answer, and a short explanation follows.

PYTHON

```
# Code Used in the session
curr_pop = 10000

for i in range(10,0,-1):
    print(i,curr_pop)
    curr_pop = curr_pop - 0.1*curr_pop
```

PYTHON

```
# Correct Answer of above question:
curr_pop = 10000

for i in range(10,0,-1):
    print(i,curr_pop)
    curr_pop /= 1.1
```

Explanation

Each year the population = previous year $\times$ 1.1 (100% + 10%). Walking back 10 years, the population is divided by 1.1 at every step. Starting from 10000, the loop prints the population for years 10 down to 1. This 'reverse the multiplier' idea also shows up in compound interest and financial projections.

Sequence sum — $1/1! + 2/2! + 3/3! + \dots$

Compute $1/1! + 2/2! + 3/3! + \dots$ up to n terms, where $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Sequence sum

What each term looks like: term $i = i / i!$.

Performance trick: do not recompute $i!$ from scratch each time — update $\text{fact} *= i$ (the previous factorial times i).

Try it: complete the `# code here` cell below.

PYTHON

```
# 1/1! + 2/2! + 3/3! + ...
n = int(input())
result = 0
fact = 1
for i in range(1, n+1):
    fact *= i          # reuse previous factorial
    result += i / fact
```

For loop vs While

FOR	WHILE
when you know the count / iterable	when a condition decides how long
iterate a list/tuple/set/dict/str/range	keep running until something changes
impossible to forget the counter	you must update the counter yourself
e.g. <code>for x in data:</code>	e.g. <code>while guess != target:</code>

Rule of thumb: if you can say 'do this for each item' $\rightarrow$ `for`. If you can only say 'do this while ...' $\rightarrow$ `while`.

Run the practice cell below.

Nested Loops

A loop **inside** a loop. The inner loop runs completely for every single pass of the outer loop.

Use cases: generating unique combinations, matrices, coordinates, and printing **patterns** (below).

The pattern rule: outer loop = rows, inner loop = what happens within one row. Combine with `end= ''` inside a row and a bare `print()` to move to the next row.

Unique-combination trick: start the inner loop at `i + 1` so pairs like (1,2) are not repeated as (2,1).

Run the code cells below (examples + unique combinations).

PYTHON

```
for i in range(1, 5):
    for j in range(1, 5):
        print(i, j)      # runs 4 x 4 = 16 times
```

Loop Control Statement

Three statements steer loops from inside:

- **Break** — exit the whole loop immediately
- **Continue** — skip the current iteration, go to the next
- **Pass** — do nothing (placeholder so an empty block is valid)

Each has its own demo and an exercise cell below.

Break

Use case — linear search: scan until you find the item, then stop early instead of walking the whole collection. Prime checking uses the same idea: `break` once you find any divisor.

Run the code cells below (break demo + prime-in-range example).

PYTHON

```
for i in range(10):
    if i == 5:
        break      # Loop STOPS at 5
```

Continue

Use case — e-commerce filtering: skip blocked/invalid items while still processing the rest of a list.

Difference from break: `continue` skips **one** iteration; `break` ends the **whole** loop.

Run the code cells below (continue demo + e-commerce example).

PYTHON

```
for i in range(10):
    if i == 5:
        continue    # SKIP 5, keep looping
```

Pass

Why use it? Python requires indented bodies; an empty (comment-free) block is a `SyntaxError`. `pass` fills the space so the code still runs while you plan the real logic — a TODO marker.

Run the code cell below (pass demo).

PYTHON

```
if x > 0:  
    pass      # TODO placeholder – valid but does nothing
```

Strings in Python

1. Creating Strings

What are strings

Immutable sequences of Unicode characters.

Why

For text representation/processing.

Key semantics

Immutable (modifications create new strings), indexable, sliceable, iterable.

Common mistakes

Assuming in-place modification works.

AI relevance

Text preprocessing for NLP.

What it is: any text between quotes is a string. Python accepts single (`'...'`), double (`"..."`), and triple quotes (`'''...'''`), which is handy when the string itself contains quotes.

How it works:

- `'hello'`, `"hello"`, `'''hello'''` and `"""hello"""` are all equal
- triple quotes keep line breaks — perfect for multi-line text
- `'it\'s raining'` — backslash escapes the inner quote
- In a string like `"it's raining outside"`, the double quotes let you keep the apostrophe without escaping

Rules: quotes around a string only matter at the start and end — what matters inside is content. There is no 'correct' quote style; choose whichever avoids escaping.

Run the cells below — one creates strings every way, the other solves the apostrophe problem.

PYTHON

```
s = 'hello'
s = "hello"
# multiline strings
s = '''hello'''
s = """hello"""
s = str('hello')
print(s)
```

PYTHON

```
"it's raining outside"
```

2. Indexing & Slicing

Indexing — grab ONE character at a position. Python indexes from **0** from the left (`s[0]` is the first char) and from **-1** from the right (`s[-1]` is the last char).

Slicing — grab a SUBSTRING. The general form is `s[start : stop : step]` — start included, **stop excluded**, step decides direction and stride.

Error: an index outside the string raises `IndexError`. Slices never do — they just shrink.

Run the cells below — positive indexing, negative indexing, and three slicing recipes.

PYTHON

```
# Positive Indexing
s = 'hello world'
print(s[41])
```

PYTHON

```
# Negative Indexing
s = 'hello world'
print(s[-3])
```

3. Strings are IMMUTABLE

Immutability means a string's content can never change once created. Python strings have no 'character slots' you can rewrite.

What happens if you try?

What you CAN do: reassign the whole variable (`s = 'Hello'`) or build a new string from parts. `del s` deletes the variable itself, not 'parts' of it — deleting a slice (`del s[-1:-5:2]`) is also a `TypeError`, because strings do not support item or slice deletion.

Why immutable? Strings are hashed and shared everywhere (dict keys, interned values) — immutability makes that safe and fast.

Run the cells below. The first deliberately errors to show the rule; the others show deletion attempts.

PYTHON

```
s = 'hello world'
s[0] = 'H'

# Python strings are immutable
```

PYTHON

```
s = 'hello world'
del s
print(s)
```

4. Operations on Strings

Strings: What/Why/Semantics

Strings are immutable sequences of Unicode chars. Immutability means operations create new strings; safe as dict keys if hashable. Slicing/indexing supported.

Common mistakes

Assuming in-place modification.

Arithmetic:

- `'delhi' + 'mumbai'` → concatenate (join) with `+`
- `'delhi' * 5` → repeat the string 5 times
- `'*' * 50` → a 50-star divider line (classic CLI trick)

Relational — strings compare **lexicographically** (dictionary order, by ASCII code):

- `'mumbai' > 'pune'` → False ('m' < 'p')
- `'Pune' > 'pune'` → False (uppercase letters come before lowercase in ASCII)

Logical (truthiness):

- `'hello' and 'world'` → 'world' (and returns the last truthy operand)
- `'hello' or 'world'` → 'hello' (or returns the first truthy operand)
- `'' and 'world'` → '' (empty string is **falsy**)
- `not 'hello'` → False

Iteration & membership:

- `for i in 'hello':` walks the string char by char
- `'D' in 'delhi'` → membership check

Run the cells below — they show each group one by one.

PYTHON

```
print('delhi' + ' ' + 'mumbai')
```

PYTHON

```
print('delhi'*5)
```

5. Common Functions — len / max / min / sorted

Built-in functions that take a string (or any iterable):

- `len('hello world')` → number of characters, INCLUDING spaces: 11
- `max('hello world')` → largest character by ASCII code: `'w'` (lowercase wins over most things)
- `min('hello world')` → smallest character: a **space** (ASCII 32 is smaller than letters)
- `sorted('hello world', reverse=True)` → returns a LIST of single characters sorted descending, e.g. `['w','r','o','o','l','l','h','e',' ',' ','d']`

Why sorted matters: it is the backbone of frequency analysis and reorderable tokens in text pipelines.

Note: strings have no 'digits as numbers' here — everything compares by raw character code.

Run the cells below — each shows one of these functions.

PYTHON

```
len('hello world')
```

PYTHON

```
max('hello world')
```

6. Case Methods — capitalize / title / upper / lower / swapcase

All four return a NEW string — the original is untouched (immutability).

- `s.capitalize()` → first char uppercase, rest lower: `'Hello world'`
- `s.title()` → first letter of every word uppercase: `'Hello World'`
- `s.upper()` → everything UPPERCASE
- `'Hello World'.lower()` → everything lowercase
- `'HeLlO WorLD'.swapcase()` → swap each char's case: `'hElLo wOrLd'`

ML connection: `lower()` is the single most-used step in text cleaning — model inputs must be case-normalized or 'Hello' and 'hello' look like different words.

Remember: `print(s.capitalize()); print(s)` shows the original still unchanged afterwards.

Run the cells below — each tests one method.

PYTHON

```
s = 'hello world'
print(s.capitalize())
print(s)
```

PYTHON

```
s.title()
```

7. Search: count / find / index

- `s.count(sub)` → how many times a substring appears: `'my name is nitish'.count('i')` → 3
- `s.find(sub)` → index of the FIRST occurrence, or -1 if missing (safe): `'...'.find('x')` → -1
- `s.index(sub)` → same as find, but raises `ValueError` if missing

Rule of thumb: use `find()` when you do not know if the pattern is present; use `index()` when its presence is guaranteed and a missing one should be treated as a bug.

Run the cells below. Notice the contrast: find returns -1 for an absent letter, index crashes with an error.

PYTHON

```
'my name is nitish'.count('i')
```

PYTHON

```
'my name is nitish'.find('x')
```

8. endswith / startswith

- `s.endswith(sub)` → True if `s` ENDS with `sub`, else False
- `s.startswith(sub)` → True if `s` STARTS with `sub`, else False

Why useful: routing logic — 'is this file a .py?', 'does this URL start with https?', 'is this message an error?'

Run the cells below to see both checks.

PYTHON

```
'my name is nitish'.endswith('sho')
```

PYTHON

```
'my name is nitish'.startswith('1my')
```

9. format()

Loops

for iterates over iterables; while runs while condition true. Use break/continue/else appropriately.

When to use

Repetitive tasks, processing collections.

9. format() — placeholders

`'...{0}...{1}...'.format(v0, v1)` — `{0}` gets replaced by the first argument, `{1}` by the second, in order of the numbers inside the braces. You may also use named fields `{name}` with keyword arguments.

The trick: the `{0}` / `{1}` numbers refer to argument position, so you can reorder, repeat, and reuse values freely.

Modern alternative: f-strings (`f'Hi {name}'`) are preferred in new code, but `format()` still appears in older codebases and templates.

Run the cell below to see the placeholder swap.

PYTHON

```
name = 'nitish'
gender = 'male'

'Hi my name is {1} and I am a {0}'.format(gender, name)
```

PYTHON

```
name = 'nitish'
gender = 'male'
'Hi my name is {1} and I am a {0}'.format(name, gender)
# -> 'Hi my name is male and I am a nitish' (note the SWAP!)
```

10. The “is...” Test Methods

All return a **bool** and check a whole-string property:

- `'nitish1234%'.isalnum()` → False (`%` is not alphanumeric; `_` and special symbols fail)
- `'nitish'.isalpha()` → True (letters only)
- `'123abc'.isdigit()` → False (contains letters — digits only would be True)
- `'first-name'.isidentifier()` → False (hyphen is not allowed in identifier names; `first_name` would be True)

Why useful: input validation — 'is this field numeric?', 'is this a valid variable name?'

Run the cells below to verify each test.

PYTHON

```
'nitish1234%'.isalnum()
```

PYTHON

```
'nitish'.isalpha()
```

I1. split / join

- `s.split(sep)` → break a string at `sep` (default: any whitespace) → a LIST of words: `'hi my name is nitish'.split()` → `['hi', 'my', 'name', 'is', 'nitish']`
- `sep.join(list)` → glue a list of strings into ONE string: `' '.join(['hi', 'my', 'name', 'is', 'nitish'])` → `'hi my name is nitish'`

Together they are a **text parser's best friends**: split to tokens for NLP, join to rebuild output lines.

Common mistake: using `.split()` on an empty or single-whitespace string — results can surprise. Always verify the list length.

Run the cells below — one splits, one joins.

PYTHON

```
'hi my name is nitish'.split()
```

PYTHON

```
" ".join(['hi', 'my', 'name', 'is', 'nitish'])
```

I2. replace

`s.replace(old, new)` replaces **every** occurrence of `old` with `new` and returns a new string.

If `old` does not occur, the string is returned unchanged (no error).

When careful: `replace` is global — to replace only the FIRST occurrence, pass a count: `s.replace(old, new, 1)`.

Run the cell below — note what happens when the text to replace does not exist.

PYTHON

```
'hi my name is nitish'.replace('nitisrgewrhgh', 'campusx')
```

PYTHON

```
'hi my name is nitish'.replace('nitish', 'campusx') # missing -> cell shows  
expectation: no match, string unchanged
```

13. strip

`s.strip()` removes leading AND trailing whitespace, tabs, and newlines. It is the first line of defense for messy input.

- `s.strip()` — both ends
- `s.lstrip()` — leading (left) only
- `s.rstrip()` — trailing (right) only

Real-world relevance: CSV and log data always carries stray spaces — strip before comparing, splitting, or hashing.

Run the cell below and watch the trailing spaces vanish.

PYTHON

```
'nitish'.strip()
```

PYTHON

```
' nitish '.strip() # 'nitish'
'nitish'.strip() # cell shows the extra spaces disappearing
```

14. Example Programs

Strategy before each program:

1. Read the problem and decide the INPUT
2. Plan the OUTPUT
3. Break the logic into small steps
4. Pick the string tools from this session
5. Test edge cases (empty strings, spaces, single letters)

The eight programs below use everything learned so far:

Program 1 — length without len()

Loop over the string with a counter: for every character, add 1. Emulates `len()` manually.

Cell below: read a string and count its characters without `len()`.

Program 2 — username from email

Everything before the `@` is the username: `'nitish24sacademy@gmail.com'` → `nitish24sacademy`.

Idea: `find('@')` gives the split point; slice `email[:pos]`, or just use `email.split('@')[0]`.

Cell below: extract the username from a given email.

Program 3 — character frequency

Count how many times a specific character appears.

Idea: `s.count(ch)`, or walk the string with a manual counter, or build a frequency dict.

Cell below: count the frequency of a given character in a string.

Program 4 — remove a character

Delete every occurrence of a given character from a string.

Idea: `s.replace(ch, '')` replaces the char with nothing — clean and one line.

Cell below: remove a particular character from a string.

Program 5 — palindrome check

A word reads the same forwards and backwards (`'madam'`).

Idea: compare `s == s[::-1]` (reverse slice), ignoring case/spaces for a stronger check.

Cell below: check whether a string is a palindrome.

Program 6 — word count without split()

Count words by counting spaces: roughly `words = spaces + 1`.

Idea: loop and count every space; handle edge cases like empty or double-spaced strings carefully.

Cell below: count words in a string WITHOUT `split()`.

Program 7 — manual title case

Uppercase the first letter of every word without using `title()`.

Idea: split into words, uppercase the first char of each, join back — or track spaces while walking the string.

Cell below: convert a string to title case manually.

Program 8 — int to string without str()

Convert an integer into its string form without calling `str()`.

Idea: repeatedly `% 10` to peel digits and `// 10` to shrink the number, then build the string from the digit characters.

Cell below: convert an integer to a string.

PYTHON

```
# Find the Length of a given string without using the Len() function

s = input('enter the string')

counter = 0

for i in s:
    counter += 1

print('length of string is',counter)
```

PYTHON

```
# Extract username from a given email.
# Eg if the email is nitish24singh@gmail.com
# then the username should be nitish24singh

s = input('enter the email')

pos = s.index('@')
print(s[0:pos])
```

Real-World and AI Engineering Relevance

Where this shows up in AI engineering

- expression evaluation and printing
- operators, precedence and truthiness
- control flow and loop control
- string methods, slicing and formatting

EXAM AUR INTERVIEW PERSPECTIVE

Agar koi concept interview ya exam mein poochha jaye, to definition ke saath apna ek chhota practical example zaroor bolo. Sirf theory batane se confidence nahi dikhta - implementation dikhane se dikhta hai.

Quick Reference

Short, representative snippets collected from the unit's notebooks. Use them as syntax reminders, not as complete programs.

PYTHON

PYTHON OUTPUT

```
# Python is a case sensitive language
print('Hello World')
```

PYTHON

1.3 THE END PARAMETER & NEWLINE BEHAVIOUR

```
print('hello')
print('world')
```

PYTHON

1.3 THE END PARAMETER & NEWLINE BEHAVIOUR

```
print('hello',end='-')
print('world')
```

PYTHON

2.1 NUMERIC TYPES — INT & FLOAT

```
# Integer
print(8)
# 1*10^308
print(1e309)
```

PYTHON

2.1 NUMERIC TYPES — INT & FLOAT

```
# Decimal/Float
print(8.55)
print(1.7e309)
```

PYTHON

2.2 BOOL & STR — TRUTH VALUES AND TEXT

```
# Boolean
print(True)
print(False)
```

PYTHON

2.2 BOOL & STR — TRUTH VALUES AND TEXT

```
# Text/String
print('Hello World')
```

PYTHON

2.3 COMPLEX, COLLECTIONS & TYPE()

```
# complex  
print(5+6j)
```

Revision Checklist

Work through this list without looking at the notes. Anything you cannot explain aloud belongs back in the revision queue.

☐ 1.1 print() — basics, quoting & case sensitivity☐ Python Output☐ 1.2 Multiple values & the sep parameter☐ 1.3 The sep & end pair☐ 1.3 The end parameter & newline behaviour☐ 2. Data Types☐ 2.1 Numeric types — int & float☐ 2.2 bool & str — truth values and text☐ 2.3 complex, collections & type()☐ 2.4 Data Types — at a glance☐ 3.1 Static vs Dynamic Typing☐ 3.2 Dynamic Binding & assignment styles☐ 4. Comments — notes inside your code☐ 5.1 Keywords — reserved words

☐ 5.2 Identifiers — naming rules☐ 6. User Input — the `input()` function☐ 7.1 Implicit Type Conversion☐ 7.2 Explicit Type Conversion (Casting)☐ 8.1 Numeric literals — binary / octal / hex / float☐ 8.2 String literals, complex literals & None☐ 8.3 Bool as int & None☐ 9. Operators — Overview